

IncidentFlow

Guide Pratique de Présentation & Démonstration Encadrant

Tracé des Fichiers Code Source & Procédure pgAdmin SQL

Objectif du Support

Ce document fournit une structure pas-à-pas pour présenter l'application **IncidentFlow** à votre encadrant. Il regroupe la liste exacte des fichiers à montrer dans le code source (du modèle de données au frontend) ainsi que les requêtes SQL prêtes à l'emploi dans **pgAdmin** pour prouver la persistance en base de données.

Application : IncidentFlow (Gestion d'Incidents Interne)
Composants : Users, Roles, Moteur de Workflow Real-time
Outil BDD : PostgreSQL 16 + pgAdmin 4
Auteur : Équipe de Développement NetMar
Date : 24 juillet 2026

Table des matières

1	Architecture Globale & Parcours des Données	2
2	Partie 1 : Les Fichiers Exacts du Code à Montrer	2
2.1	Étape 1 : Modélisation des Données (Base de Données vers Java JPA)	2
2.2	Étape 2 : Logique Métier & Traitement des Entités (Service Layer)	3
2.3	Étape 3 : Points d'Entrée API REST (Controllers Backend)	3
2.4	Étape 4 : Interface Réactive Frontend (React 19 / Vite)	3
3	Partie 2 : Guide Pratique pgAdmin & Requêtes SQL	3
3.1	Paramètres de Connexion PostgreSQL dans pgAdmin	3
3.2	Procédure d'Exécution des Requêtes SQL	3
3.3	Requêtes SQL de Démonstration	4
3.3.1	1. Afficher les Rôles :	4
3.3.2	2. Afficher les Utilisateurs avec leur Rôle rattaché :	4
3.3.3	3. Afficher le Workflow Actif :	4
3.3.4	4. Afficher tous les États du Workflow :	4
3.3.5	5. Afficher les Transitions Autorisées :	4
3.4	Scénario de Démonstration "Live" devant l'Encadrant	4

1 Architecture Globale & Parcours des Données

Lors de votre présentation, vous pouvez expliquer le schéma suivant qui résume le passage des données des utilisateurs, des rôles et des workflows :

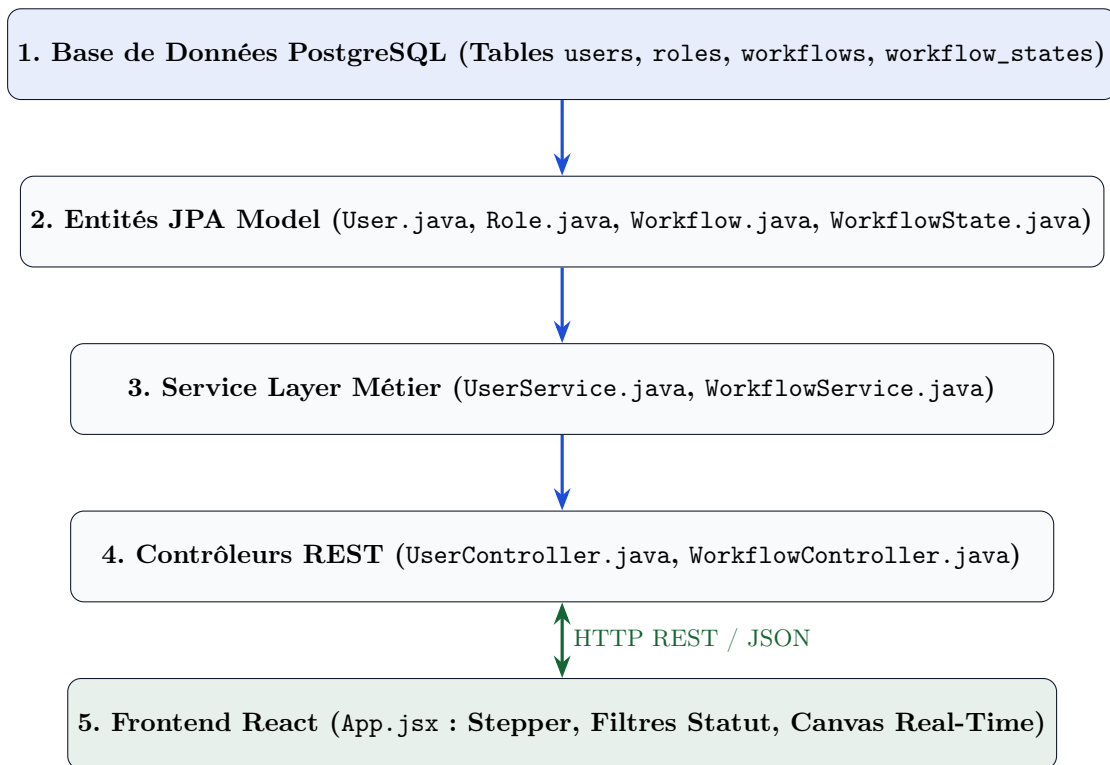


FIGURE 1 – Chaîne de transmission des données dans IncidentFlow

2 Partie 1 : Les Fichiers Exacts du Code à Montrer

Suivez cette séquence dans votre IDE (VSCode / IntelliJ) pour montrer exactement où passent les données.

2.1 Étape 1 : Modélisation des Données (Base de Données vers Java JPA)

Montrez à votre encadrant les classes annotées avec Hibernate :

- 1. Les Rôles :** `backend/src/main/java/com/netmar/incidentflow/model/Role.java`
- 2. Les Utilisateurs :** `backend/src/main/java/com/netmar/incidentflow/model/User.java`
— Montrez l'annotation `@ManyToOne` reliant chaque utilisateur à son rôle.
- 3. Le Workflow Racine :** `backend/src/main/java/com/netmar/incidentflow/model/Workflow.java`
— Montrez l'annotation `@OneToMany(mappedBy = "workflow", cascade = CascadeType.ALL, orphanRemoval = true)` sur les collections d'états et de transitions.
- 4. Les États du Workflow :** `backend/src/main/java/com/netmar/incidentflow/model/WorkflowState.java`
- 5. Les Transitions du Workflow :** `backend/src/main/java/com/netmar/incidentflow/model/WorkflowTransition.java`
- 6. L'Initialisation en Base au Démarrage :** `backend/src/main/java/com/netmar/incidentflow/config/DatabaseInitializer.java`
— Montrez la méthode `run()` qui peuple la base PostgreSQL au lancement avec les rôles par défaut et le *Workflow Standard*.

2.2 Étape 2 : Logique Métier & Traitement des Entités (Service Layer)

1. `UserService.java` (`backend/src/main/java/com/netmar/incidentflow/service/UserService.java`)
— Montrez les méthodes `getAllUsers()`, `getAllRoles()` et `getCurrentUser()`.
2. `WorkflowService.java` (`backend/src/main/java/com/netmar/incidentflow/service/WorkflowService.java`)
— Montrez la méthode `getAllWorkflows()`.
— **Point Clé** : Montrez la méthode `saveWorkflow()` et expliquez comment l'application charge l'entité gérée par Hibernate (`findById`), vide les anciennes collections (`.clear()`) puis ré-attache les nouveaux états pour éviter les erreurs d'entités détachées JPA.

2.3 Étape 3 : Points d'Entrée API REST (Controllers Backend)

1. `WorkflowController.java` (`backend/src/main/java/com/netmar/incidentflow/controller/WorkflowController.java`)
— Montrez `GET /api/workflows` et `PUT /api/workflows/{id}`.
2. `UserController.java` (`backend/src/main/java/com/netmar/incidentflow/controller/UserController.java`)
— Montrez `GET /api/users` et `GET /api/roles`.
3. `IncidentController.java` (`backend/src/main/java/com/netmar/incidentflow/controller/IncidentController.java`)
— Montrez la méthode de suppression réservée à l'Admin : `DELETE /api/incidents/{code}`.

2.4 Étape 4 : Interface Réactive Frontend (React 19 / Vite)

Ouvrez le fichier `frontend/src/App.jsx` :

1. **Requêtes Async** : Montrez les fonctions `fetchWorkflows()` et `fetchUsers()` appelées dans le hook `useEffect`.
2. **Variables d'État React** : Montrez `activeWorkflow`, `workflows` et `usersList`.
3. **Rendu en Temps Réel** :
 - Le menu déroulant du filtre de statut dans la liste des incidents s'alimentant depuis `activeWorkflow.states`.
 - Le **Stepper horizontal d'avancement** (`workflow-stepper-container`).
 - Le **Panneau des boutons d'actions disponibles** (`workflow-actions-panel`).

3 Partie 2 : Guide Pratique pgAdmin & Requêtes SQL

3.1 Paramètres de Connexion PostgreSQL dans pgAdmin

Dans **pgAdmin 4**, créez ou ouvrez la connexion à la base de données avec les identifiants du projet :

Identifiants de Connexion pgAdmin

- **Host name / address** : `localhost` (ou `127.0.0.1`)
- **Port** : `5433` (*Port PostgreSQL exposé par Docker Compose*)
- **Maintenance database** : `IncidentFlow_db`
- **Username** : `IncidentFlow_user`
- **Password** : `incidentflow_secure_db_pass`

3.2 Procédure d'Exécution des Requêtes SQL

1. Dans l'arborescence gauche de pgAdmin, développez **Servers** → **IncidentFlow_db**.
2. Faites un clic droit sur la base **IncidentFlow_db** et sélectionnez **Query Tool** (ou *Outils* → *Outil de requête*).
3. Saisissez et exécutez les requêtes SQL ci-dessous (touche **F5**).

3.3 Requêtes SQL de Démonstration

3.3.1 1. Afficher les Rôles :

```
SELECT id, name, description
FROM roles;
```

Listing 1 – Consultation des rôles en base de données

3.3.2 2. Afficher les Utilisateurs avec leur Rôle rattaché :

```
SELECT u.id, u.name, u.email, u.post, u.department, r.name AS role_nom
FROM users u
LEFT JOIN roles r ON u.role_id = r.id;
```

Listing 2 – Jointure entre Utilisateurs et Rôles

3.3.3 3. Afficher le Workflow Actif :

```
SELECT id, name, category, active
FROM workflows;
```

Listing 3 – Consultation de l'entité Workflow

3.3.4 4. Afficher tous les États du Workflow :

```
SELECT s.id, s.name, s.label, s.color_class, w.name AS workflow_nom
FROM workflow_states s
JOIN workflows w ON s.workflow_id = w.id
ORDER BY s.id ASC;
```

Listing 4 – Consultation des états rattachés au workflow

3.3.5 5. Afficher les Transitions Autorisées :

```
SELECT t.id, t.from_state, t.to_state, t.role_required, w.name AS workflow_nom
FROM workflow_transitions t
JOIN workflows w ON t.workflow_id = w.id;
```

Listing 5 – Consultation du graphe de transitions du workflow

3.4 Scénario de Démonstration "Live" devant l'Encadrant

Déroulé du Scénario de Démonstration

1. **Étape A (Web)** : Dans l'interface web React, allez dans l'onglet "*Gestion du Workflow*".
2. **Étape B (Web)** : Ajoutez un nouvel état (ex : "*Validation SR*") et enregistrez.
3. **Étape C (pgAdmin)** : Basculez sur **pgAdmin** et exécutez la requête SQL des états :

```
SELECT id, name, label FROM workflow_states ORDER BY id DESC;
```
4. **Résultat** : Montrez à votre encadrant la nouvelle ligne "*Validation SR*" insérée en temps réel dans la table PostgreSQL!